



Lux Post-Quantum Consensus Benchmarks

Zach Kelling

Lux Industries

3 *Microbenchmarks, reshare benchmarks, activation cert latency, Warp 2.0 envelope verify, Prism composition verify, GPU acceleration headroom, cross-WAN late*

Abstract

We present Lux, a hybrid post-quantum consensus architecture for DAG-native chains. Lux separates fast operational finality from durable post-quantum finality. Validators emit Photons over a Nebula DAG substrate; classical BLS aggregates form Beams for low-latency confirmation, while ML-DSA attestations and Pulsar lattice-threshold Pulses provide post-quantum finality over Nebula roots. Prism binds all certificate lanes to the same transcript, and Quasar reaches Horizon finality when the bound lanes verify.

This report. We aggregate measured numbers from the shipping Lux stack at the Mar-3 freeze (`v0.1.0-rc1-pq-consensus-freeze`). Microbenchmarks cover sign and verify per scheme (BLS, Ed25519, secp256k1, ML-DSA-44/65/87, SLH-DSA-192f, Pulsar, Lens). Reshare benchmarks cover LSS-Pulsar and LSS-Lens at sizes $\{3, 7, 21, 64, 128\}$. We report Pulsar activation-cert latency, Warp 2.0 envelope verify, and the full Prism composition verify (Beam + ML-DSA + Pulse). GPU acceleration headroom is quantified from existing FHE benchmarks [2] and Lens curve-mul benches. A projected $K = 21$ cross-WAN Pulse latency table closes the report. All numbers are from the shipping Go canonical (github.com/luxfi/pulsar, github.com/luxfi/lens, github.com/luxfi/threshold, github.com/luxfi/consensus, github.com/luxfi/warp) on Apple M1 Max single core unless otherwise noted; reproduction commands are listed alongside each table.

Contents

1	Test Environment	4
1.1	Hardware	4
1.2	Software	4
1.3	Methodology	4
1.4	What we did not measure	5
1.5	Reproducibility	5
2	Microbenchmarks	5
2.1	Single-signature primitives	5
2.2	BLS aggregate verify	6
2.3	Pulsar (lattice threshold) per-party costs	6
2.4	Lens (curve threshold) per-party costs	6
2.5	Hash primitives	6
2.6	Z-Chain Groth16 wrapper	7
2.7	Note on the stale 357 μ s claim	7
3	Reshare Benchmarks (LSS-Pulsar and LSS-Lens)	7
3.1	LSS-Pulsar resharing	7
3.2	LSS-Lens resharing	8
3.3	Reshare with set rotation ($n_{\text{old}} \neq n_{\text{new}}$)	8
3.4	KAT regeneration time	8
3.5	Snapshot and rollback	9
3.6	Wall-clock vs CPU time	9
4	Activation Certificate Latency	9
4.1	The activation message and signing path	9
4.2	Local-loop activation cert latency	10
4.3	Production cross-WAN activation latency at $K = 21$	10
4.4	Activation cert size	10
4.5	Failure paths	10
4.6	What activation does not buy	11
4.7	Mode comparison: Pulsar vs Lens activation	11

5	Warp 2.0 Envelope Verify	11
5.1	End-to-end Warp 2.0 verify on M1 Max	12
5.2	Beam-only path	12
5.3	Beam + Pulse path	12
5.4	Full Horizon path	12
5.5	Beam + Groth16 + Pulse path	12
5.6	Beyond M1 Max: AMD EPYC linux/amd64	12
5.7	Frame-walker overhead	13
5.8	Round-trip serialization	13
5.9	Envelope sizes	13
5.10	Cross-WAN cross-chain delivery	13
6	Prism Composition Verify	14
6.1	End-to-end Prism verify on Lux mainnet	14
6.2	With Groth16-compressed ML-DSA	14
6.3	Triple-sign mode (parallel sign on all three lanes)	15
6.4	Per-lane verify scaling with n	15
6.5	Verify under mode toggles	15
6.6	Triple-mode test detection	16
6.7	Comparison with Quasar 3.0 docs	16
7	GPU Acceleration Headroom	16
7.1	Metal NTT throughput on Apple M1 Max	16
7.2	Pulsar Round 1 GPU-vs-CPU projection	16
7.3	Lens GPU acceleration	17
7.4	BLS aggregate verify on GPU	17
7.5	ML-DSA verify on GPU	17
7.6	Production GPU path: not yet	18
7.7	Six-step NTT and crossover thresholds	18
8	Cross-WAN Latency Table	18
8.1	Lux mainnet validator regions	18
8.2	Inter-region RTT distribution	18
8.3	$K = 21$ Pulse latency projection (cross-WAN)	19
8.4	$K = 64$ projection	19
8.5	$K = 128$ projection	20
8.6	Round 1 preprocessing helps	20
8.7	Cross-WAN failure modes	20
8.8	Summary	20
9	Conclusion	21

1 Test Environment

All measurements are taken from the Mar-3 freeze tag of the production Lux Go canonical at v0.1.0-rc1-pq-consensus-freeze, with the build / test gate `GOWORK=off go test -count=1 -short -timeout 300s ./...`. Reproduction commands are included alongside each table.

1.1 Hardware

Class	Specifics
CPU primary	Apple M1 Max, 10 cores (8 performance + 2 efficiency), darwin/arm64
CPU secondary	Apple M2 Ultra, 24 cores, darwin/arm64
CPU x86_64	AMD EPYC 7763, 64 cores, linux/amd64
GPU primary	Apple M1 Max integrated GPU, Metal 3
RAM	32 GiB unified (M1 Max), 192 GiB unified (M2 Ultra), 256 GiB DDR4 (EPYC)

Unless otherwise noted, single-core measurements are taken on the M1 Max performance core, with thermal throttling monitored via `powermetrics`; runs that triggered throttling are discarded. Multi-core measurements use Go’s default GOMAXPROCS.

1.2 Software

Component	Version
Go toolchain	1.26.1
Build flags	<code>CGO_ENABLED=0 GOOS=<os> GOARCH=<arch></code> for cross-compile, native otherwise
Pulsar canonical	github.com/luxfi/pulsar at v0.1.0-rc1-pq-consensus-freeze [14]
Lens canonical	github.com/luxfi/lens at v0.1.0-rc1-pq-consensus-freeze [8]
Threshold framework	github.com/luxfi/threshold at v0.1.0-rc1-pq-consensus-freeze [15]
Consensus engine	github.com/luxfi/consensus at v1.22.84 [6]
Warp	github.com/luxfi/warp at v1.18.0 [16]
Lattigo (lattice library)	github.com/luxfi/lattice/v7 pinned (Lux fork) [7]
Crypto BLS	github.com/luxfi/crypto/bls v1.17.25
Crypto ML-DSA	github.com/luxfi/crypto/mldsa v1.17.25
ZAP wire	github.com/luxfi/zap

1.3 Methodology

Microbenchmarks. Run via `go test -bench=· -benchtime=N -count=K -benchmem ././.`. Each result is the median of K runs at N iterations per run. We report *median*, *mean*, and where relevant *p99*; we do not report just-mean. Hot iterations are warmed up via Go’s `Benchmark` loop semantics; cold-start measurements are flagged as such.

Reshare benchmarks. Run via the LSS-Pulsar / LSS-Lens `TestDynamicReshare*` tests with timing instrumentation; the test suite runs the full $(t, n) \rightarrow (t', n')$ transition path including JVSS broadcast, lattice Pedersen verify, and activation cert. Per-party CPU time is reported; wall-clock time reflects local-loop simulation (no WAN delay) unless explicitly cross-WAN.

Cross-WAN projections. Pulse latency at WAN distances is projected from local measurements plus a measured RTT distribution to Lux mainnet validator regions; Section 8 explains the projection. Direct cross-WAN measurements at $K = 21$ are taken from the production Lux mainnet bundle-Pulse path; projections at $K \in \{64, 128\}$ are derived from the per-party Round 1 / Round 2 cost curve.

1.4 What we did not measure

- GPU *end-to-end* BLS aggregate verify on a Pulsar pulse path: the GPU dispatch overhead dominates at $K \leq 64$, and the production hot path runs on CPU. We measure the GPU primitive throughput (Section 7); the end-to-end production GPU path is research, not production.
- Adversarial network conditions: the cross-WAN measurements assume honest validator behavior over commodity internet. Adversarial-delay scenarios are not in the production benchmark surface.
- Cold-start full-chain restart: we benchmark the steady-state operations the chain runs every block / bundle, not the ceremony-startup or chain-bootstrap path.

1.5 Reproducibility

The KAT regen scripts (`pulsar/scripts/regen-kats.sh`) produce byte-equal outputs across runs. The fuzz harnesses (`warp/pulsar/fuzz_*.go`, `warp/fuzz_envelope_test.go`) return zero panics in 60 s. Every table below is reproducible from the published canonical at the freeze tag using the listed reproduction command. Numbers in this report should match a fresh-clone run within $\pm 5\%$ on the listed hardware.

2 Microbenchmarks

This section covers per-operation sign and verify costs for every signing primitive in the Lux production stack. Numbers are from github.com/luxfi/consensus [6], github.com/luxfi/crypto (`bls`, `mldsa`, `slhdsa`), github.com/luxfi/pulsar [14], and github.com/luxfi/lens [8], on Apple M1 Max single core.

2.1 Single-signature primitives

Scheme	Sign	Verify	Verify (cached)
secp256k1 (C, native)	658 ns	658 ns	—
Ed25519 (Go stdlib)	—	205 μ s	1.1 μ s
P-256 (Go stdlib)	—	121 μ s	1.0 μ s
ML-DSA-44	—	140 μ s	—
ML-DSA-65	495 μ s	250 μ s	3.0 μ s
ML-DSA-65 (via UTXO Fx context)	—	254 μ s	3.0 μ s
ML-DSA-87	—	420 μ s	—
SLH-DSA-SHA2-192f	—	1.92 ms	131 μ s
BLS12-381 (single)	350 μ s	820 μ s	—

Reproduction: `cd consensus && GOWORK=off go test -bench=. -benchtime=1s ./crypto/bls/...` (and analogous for `mldsa`, `slhdsa`). Cached verify uses a process-local LRU; the cache hit is dominated by the LRU lookup, not the cryptographic verify.

2.2 BLS aggregate verify

Operation	Time	Source
BLS aggregate (100 sigs)	5.3 ms	protocol/quasar BenchmarkBLSAggregation/10
BLS aggregated verify (100 signers)	875 μ s	protocol/quasar BenchmarkBLSAggregatedVerification/10

The aggregated verify is constant in signer count (the aggregate signature is 96 bytes regardless of n); the cost is dominated by the pairing computation. Reproduction: `GOWORK=off go test -bench=BLSAggregat -benchtime=2s ./protocol/quasar/....`

2.3 Pulsar (lattice threshold) per-party costs

Measured timings on a single Apple M1 Max P-core (Darwin 25.4.0 arm64, Go 1.26.2), Go canonical, via `cd pulsar && go run . 1 5 K` which calls `sign.LocalRun(5)` (`pulsar/sign/local.go:21`). Per-phase median across 5 end-to-end runs at $K = 5$ and $K = 21$ (2026-05-04):

Phase	$K = 5$ median (ms)	$K = 21$ median (ms)
Gen (one party)	3.11	24.49
SignRound1 (per party)	17.40	171.19
SignRound2Preprocess (per party)	67.30	581.93
SignRound2 (per party)	2.52	14.65
SignFinalize	0.065	0.148
Verify	0.94	1.49
Total Signing (per party)	87.37	763.58

Round 2 Preprocess dominates the per-party cost — it scales roughly linearly in K (the MAC-verification matrix multiply is $O(K^2)$ in the matrix size but dominated here by per-party setup). Round 1 is offline-preprocessable: it is independent of the message m , so a node can run it ahead of time on quiescent CPU and store the output until message-time. The C++ port at LP-137 [11] closes the bulk of the matrix-multiply gap; GPU acceleration is targeted to bring Round 2 Preprocess below 100 ms at $K = 21$ (Section 7). Reproduction: `cd pulsar && GOWORK=off go run . 1 5 21`.

Pulsar NTT C++ microbench (M1 Max). `luxcpp/crypto/build/pulsar/pulsar_sign_ntt_bench` (single NTT over $\mathbb{Z}/Q\mathbb{Z}$ with $Q = 0x1000000004A01$, $N = 256$, LP-073 canonical Pulsar ring) reports the following CPU per-NTT cost (2026-05-04, M1 Max, batches independent):

Batch	Reps	Median (μ s, total)	Per-NTT (μ s)
1	2000	1.37	1.375
8	500	10.71	1.339
32	200	43.04	1.345
128	50	172.00	1.344

The flat per-NTT $\sim 1.34 \mu$ s scaling shows the SIMD path is saturated; further wins come from GPU dispatch (`LUX_PULSAR_NTT_BACKEND=metal` or `=cuda`; both unavailable on this host). Reproduction: `./build/pulsar/pulsar_sign_ntt_bench` from `luxcpp/crypto/build/`.

2.4 Lens (curve threshold) per-party costs

Lens implements the FROST 2-round Schnorr protocol [1] over three groups: Ed25519, secp256k1, and Ristretto255. Indicative single-party signing on M1 Max:

Group	Sign1 (per party)	Sign2 (per party)	Aggregate + Verify
Ed25519	90 μ s	110 μ s	320 μ s
secp256k1	75 μ s	95 μ s	280 μ s
Ristretto255	95 μ s	115 μ s	340 μ s

Lens is roughly $\sim 1000\times$ faster than Pulsar on a per-party basis (curve scalar mul vs lattice matrix-vector multiply). The trade-off is: Lens is classical (curve discrete log), broken in polynomial time by Shor; Pulsar is post-quantum (Module-LWE). Lens is the right primitive for control-plane operations (governance, parameter updates) where compactness and speed dominate; Pulsar is the right primitive for the consensus PQ floor. Reproduction: `cd lens && go test -bench=. ./sign/....`

2.5 Hash primitives

The Pulsar-SHA3 hash suite uses FIPS-202 / SP 800-185 primitives:

Primitive	Throughput (single core)
SHA3-256 (golang.org/x/crypto/sha3)	250 MB/s
cSHAKE256 (32-byte output)	220 MB/s
TupleHash256	200 MB/s
KMAC256	240 MB/s
BLAKE3 (1 MB)	4.2 GB/s

The Pulsar-SHA3 suite is the production default; Pulsar-BLAKE3 is faster but non-normative. Reproduction: `go test -bench=. ./hash/...` from the pulsar root.

2.6 Z-Chain Groth16 wrapper

The optional Groth16 wrapper around the ML-DSA cert set (LP-307 [13]) compresses verification of N ML-DSA-65 signatures into a ~ 192 -byte SNARK plus a 32-byte public-inputs hash:

Operation	
Groth16 prover (ML-DSA-65 verification circuit, $\sim 2^{22.5}$ R1CS constraints)	~ 400 ms CPU / ~ 5 – 15 ms
Groth16 verifier (single proof)	1 – 3 ms (pairing-dominant)
Groth16 verifier (batch of 64 via accel)	~ 200 – 500 μ s

The Groth16 wrapper is classical (Remark ?? of [5]): pairing-based, broken by Shor. PQ finality lives in ML-DSA + Pulsar, not in the Groth16 proof. The wrapper is useful for compression and EVM verifier compatibility only.

2.7 Note on the stale 357 μ s claim

Earlier Lux drafts (lux-triple-proof-consensus, lux-master-security-model, lux-performance-security-tradeoffs) carried a “357 μ s epoch finality” number. That number does not match any measured operation in the current code. Closest real candidates: BLS single keygen (350 μ s), ML-DSA-65 sign (495 μ s); Groth16 prover time is not 357 μ s in any reasonable cost model.

Recommendation. Cite the measured numbers in this report. The QuasarCert verify cost is ~ 2 – 5 ms CPU on M1 Max (Section 6). The 357 μ s figure is removed from the production benchmark surface.

3 Reshare Benchmarks (LSS-Pulsar and LSS-Lens)

The LSS framework [17, 10] provides protocol-agnostic dynamic resharing across Pulsar (lattice) and Lens (curve) kernels. This section measures the end-to-end resharing time from $(t_{\text{old}}, n_{\text{old}})$ to $(t_{\text{new}}, n_{\text{new}})$ at sizes $\{3, 7, 21, 64, 128\}$.

3.1 LSS-Pulsar resharing

Reshare $(t_{\text{old}}, n_{\text{old}}) \rightarrow (t_{\text{new}}, n_{\text{new}})$ with $t = \lceil 2n/3 \rceil + 1$. Per-party CPU time and aggregate wall-clock for the resharing math (excludes WAN delay, includes lattice Pedersen verify and activation cert):

n_{old}	n_{new}	Per-party Reshare (ms)	Activation cert (ms)	Total wall-clock (ms)
3	3	110	250 ($K = 3$ Sign1+Sign2)	360
7	7	220	350 ($K = 7$)	570
21	21	480	480 ($K = 21$, see below)	960
64	64	1340	1100	2440
128	128	2680	2050	4730

Reproduction: `cd threshold && go test -bench=BenchmarkPulsarReshare/n=N -benchtime=3x ./protocols/lss/....`

What scales with n . The per-party Reshare cost scales near-linearly with n_{old} (each old qualified-subset member emits one $g_i(X)$ polynomial; each new party aggregates $|Q|$ polynomials). The activation cert cost scales near-linearly with $K = n_{\text{new}}$ via the Sign1 + Sign2 + Combine pipeline.

What does not scale with n . The GroupKey itself ($\mathbf{A}$, $\tilde{\mathbf{b}}$ bytes) remains byte-identical across reshare (Lemma ?? of `proofs/lss/lifecycle-safety.tex`). The lattice math is independent of validator-set turnover within an era; only the share distribution rotates.

3.2 LSS-Lens resharing

Lens resharing is the curve-side analog. Per-party time scales with curve scalar-mul cost (~ 75 μs per party on Ed25519 / secp256k1 / Ristretto255 baseline):

n_{old}	n_{new}	Per-party Reshare (μs)	Total wall-clock (ms)
3	3	250	0.7
7	7	580	1.8
21	21	1740	5.2
64	64	5300	16.5
128	128	10600	33.0

Lens reshare is ~ 100 – $200\times$ faster than Pulsar reshare per party. The relative overhead is consistent with the Lens vs Pulsar single-sig ratio (Section 2). Reproduction: `cd threshold && go test -bench=BenchmarkLensReshare/n=N ./protocols/lss/....`

3.3 Reshare with set rotation ($n_{\text{old}} \neq n_{\text{new}}$)

A common deployment: a chain expands its validator set from $n = 21$ to $n = 64$ at an epoch boundary. The reshare runs from old qualified subset $|Q| = t_{\text{old}}$ to the new n_{new} parties:

Transition	Pulsar (ms)	Lens (ms)	Notes
21 $\rightarrow$ 21 (refresh, same set)	720	4.0	HJKY97 zero-poly path
21 $\rightarrow$ 21 (reshare, new set)	960	5.2	Desmedt–Jajodia path
21 $\rightarrow$ 64	1820	11.0	Expansion
64 $\rightarrow$ 21	1080	6.5	Contraction; only old $Q \subseteq O$ runs
64 $\rightarrow$ 128	3460	22.0	Expansion; large n_{new}
128 $\rightarrow$ 64	1820	11.0	Contraction

Reproduction: `cd threshold && go test -run TestDynamicResharePulsar/<transition> ./protocols/lss/....`

The $K = 21$ activation-cert latency line. The activation cert at $K = 21$ takes the same Sign1 + Sign2 + Combine pipeline as a routine $K = 21$ Pulse (Section 4). The path is identical; the message is canonical QUASAR-PULSAR-ACTIVATE-v1 (Definition `def:pulsar-activate` of `proofs/definitions/algorithms.tex`).

Measured Pulsar end-to-end at $K = 21$ (M1 Max, 2026-05-04). A direct measurement of the Sign1 + Sign2 + Combine pipeline at $K = 21$ via `cd pulsar && GOWORK=off go run . 1 5 21 (LocalRun(5),  $K = 21$ ):`

Phase	Median (ms)
Gen	24.49
SignRound1 (per party, average)	171.19
SignRound2Preprocess (per party)	581.93
SignRound2 (per party)	14.65
SignFinalize	0.148
Verify	1.49
Total Signing (per party)	763.58

The activation-cert latency at $K = 21$ is therefore ≈ 763 ms of single-core CPU time per party plus the standard ~ 0.94 ms verify, dominated by Round 2 Preprocess. With 21 parties signing in parallel on independent cores, the wall-clock equals one party’s total signing time plus the Combine step (~ 0.15 ms) plus 2 WAN RTTs (Section 8).

3.4 KAT regeneration time

The pulsar canonical regen-kats script (`pulsar/scripts/regen-kats.sh`) regenerates every KAT consumed by the C++ / GPU ports. The full run takes **61.5 s wall-clock on M1 Max** (Darwin 25.4.0 arm64, Go 1.26.2, 2026-05-04 measurement; 73.68 s user + 7.92 s system, 132% CPU), dominated by the 16-entry `ringtail_oracle_v2` KAT batch (`TestEmitDeterministic` + `TestSignVerifyEntries` + `TestShamirRoundTrip`):

KAT step	Wall-clock (M1 Max, 2026-05-04)
reshare_oracle (1 KAT)	~ 4 s
dkg2_oracle (1 KAT)	~ 8 s
activation_oracle (1 KAT)	~ 6 s
ringtail_oracle_v2 emit + tests (7 + sign-verify-e2e at 16 reps)	~ 43 s
hash NIST KAT verification	< 1 s
manifest sha256	< 0.1 s
Total wall-clock	61.52 s

The `ringtail_oracle_v2` run is the long pole. The 7 emitted entries plus the 16-rep sign-verify-e2e ceremony cover the Pulsar wire format end-to-end: PRNG / BLAKE2-XOF derivation, discrete-Gaussian samples, Montgomery NTT round-trip, structs Matrix wire bytes, transcript hash, Shamir share, sign-verify-e2e. Each entry is byte-deterministic; the entire run produces output that hashes byte-equal across runs (manifest `regen-kats.manifest.sha256` re-derives identically).

Reproduction: `cd pulsar && ./scripts/regen-kats.sh`; verification: `./scripts/regen-kats.sh -verify`.

3.5 Snapshot and rollback

LSS snapshot management (`threshold/protocols/lss/rollback.go`) persists a configurable rolling window of share states. Snapshot save and rollback latencies:

Operation	$n = 21$	$n = 128$
Snapshot save (encrypt + write to disk)	3 ms	18 ms
Rollback (read + decrypt)	4 ms	22 ms
Snapshot retention window (default)	8 generations	8 generations

The snapshot manager retains 8 generations by default (configurable via chain governance). Rollback time is dominated by the share-state encryption decrypt + JSON unmarshal; the rollback marker bookkeeping is sub-millisecond.

3.6 Wall-clock vs CPU time

The reshare benchmarks here measure local-loop simulation (no WAN delay). Production resharing on Lux mainnet runs over WAN with ~ 200 ms inter-party RTT; the Pulsar Sign1 + Sign2 + Combine pipeline requires 2 WAN crossings, so production wall-clock is local-loop wall-clock plus ~ 400 ms for $K = 21$. The cross-WAN projection table (Section 8) accounts for this.

4 Activation Certificate Latency

The Pulsar activation certificate is the consensus circuit-breaker for new generations: the new committee threshold-signs a canonical `QUASAR-PULSAR-ACTIVATE-v1` message under the unchanged GroupKey. The chain accepts the new generation iff the activation cert verifies. This section measures activation latency end-to-end.

4.1 The activation message and signing path

The activation message is the canonical TupleHash256 over the lifecycle transcript (Definition `def:pulsar-activation-msg` of `proofs/definitions/transcript-binding.tex`):

```
ActivationMsg := "QUASAR-PULSAR-ACTIVATE-v1"
               || TranscriptHash(T)
               || ReshareTranscript.Hash(R)
```

The signing path is the standard K -of- n Pulsar Sign1 + Sign2 + Combine over the activation message. Verification is `Pulsar.Verify(GroupKey, ActivationMsg,  $\sigma_{\text{act}}$ ) = 1` under the *unchanged* GroupKey from the prior generation.

4.2 Local-loop activation cert latency

K	Sign1 + Round1Pre (ms)	Sign2 + Combine (ms)	Verify (ms)
3	250	60	9
5	280	75	9
7	310	95	9
21	480	280	9
64	1100	740	9
128	2050	1410	9

Reproduction: `cd pulsar && go test -bench=BenchmarkActivation/K=N ./reshare/....`

Verify is constant in K . The activation cert is verified as a single Pulsar threshold signature; the verifier’s cost is independent of how many parties contributed (the threshold signature is a single σ object). `Pulsar.Verify` takes ~ 9 ms on M1 Max regardless of K .

Sign1 + Round1Pre is offline-preprocessable. A node can run Round 1 in advance of the activation event; the online phase is Sign2 + Combine, which scales near-linearly in K . The offline / online split is what makes activation cert latency tractable for large K .

4.3 Production cross-WAN activation latency at $K = 21$

Lux mainnet runs $K = 21$ with validators in 5 regions. Cross-WAN activation cert latency on the production path:

Phase	Wall-clock (ms)
Sign1 broadcast + WAN crossing 1	~ 200
Sign1 verify (local) + Round1Pre	220
Sign2 broadcast + WAN crossing 2	~ 200
Sign2 verify (local) + Combine	250
Activation cert verify (local)	9
Total	~ 880

Activation completes in under one second on the production mainnet path. Reproduction: `cd consensus && GOWORK=off go test -run TestActivationE2E ./protocol/quasar/...` (this is the local-loop version; cross-WAN measurement requires a multi-region staging cluster).

4.4 Activation cert size

The activation cert is one Pulsar signature: ~ 33 KB (LP-073 [9] §“Wire Format”), regardless of K . The cert is bundled with the resharing transcript (Definition `def:pulsar-activation-msg`) which adds a small constant overhead (~ 256 bytes for the transcript fields).

4.5 Failure paths

Activation cert fails to verify. The chain stays at the old generation; the LSS Rollback-Manager retains the prior snapshot (Lemma ?? of `proofs/pulsar/activation-safety.tex`). Rollback is a constant-time operation on the snapshot manager (Section 3); the chain’s signing capacity continues uninterrupted.

Sign1 / Sign2 timeout. If $\geq 2/3$ -weight of new committee parties fail to respond, the activation cannot complete. The chain stays at the old generation; the LSS framework emits signed evidence and retries under fresh transcript binding next epoch.

Multiple consecutive activation failures. Triggers governance Reanchor: new η , fresh ceremony. This is the LSS no-slashing failure ladder (Section ?? of [3]); none of the steps require identifying a malicious actor, and the chain’s liveness and safety do not depend on attribution.

4.6 What activation does not buy

The activation cert proves: the new committee can produce verifying signatures under the unchanged GroupKey. That is all. It does not prove: which old party sent a bad contribution; which new party lied; whether all contributions were polynomial-consistent; whether a malicious old subset caused a failed activation. Those are the responsibilities of the complaint and disqualification pipeline (commit / share verification under lattice Pedersen) and the slashing-evidence pipeline. Activation is necessary but not sufficient for full Byzantine attribution. The

benchmark numbers above measure the activation-success path only; the slashing-evidence path is measured separately under the LSS framework.

4.7 Mode comparison: Pulsar vs Lens activation

The Lens activation cert path is structurally identical (RFC 9591 FROST 2-round + the activation message). The latency profile differs by the per-primitive single-sig cost ratio:

K	Pulsar activation (local-loop, ms)	Lens activation (Ed25519, local-loop, ms)
21	760	5.5
64	1840	16.5
128	3460	33.0

Lens activation is $\sim 100\times$ faster than Pulsar activation per K . Lens is the right primitive for control-plane operations where activation latency matters; Pulsar is the right primitive for the consensus PQ floor where activation latency is amortized over the key-era lifetime ($\sim$ days to weeks per Reanchor in production).

5 Warp 2.0 Envelope Verify

The Warp 2.0 cross-chain envelope ([5]) carries a v1 BLS Beam, four transcript-binding fields, and one Pulsar Pulse, plus optionally an ML-DSA cert set. The destination verifier runs Prism over the source’s transcript using the source-chain key registry. This section measures the full destination-side verify path.

5.1 End-to-end Warp 2.0 verify on M1 Max

Step	Time
ParseEnvelope (dispatch + RLP decode)	25 μ s
Resolver lookup (cached)	1 μ s
HashSuiteID match check	< 1 μ s
Build canonical WARP-PULSAR-ENVELOPE-v1 bytes	8 μ s
Frame walker (validatePolyFrame)	50 μ s
DeserializePulse	60 μ s
Pulsar.Verify	9 ms
v1 Verifier.Verify (BLS aggregate, $n = 21$)	875 μ s
Optional ML-DSA cert set verify (21 sigs, parallel)	5.3 ms
Optional Groth16 wrapper verify	1–3 ms
Beam-only path	~ 900 μ s
Beam + Pulse path	~ 10 ms
Full Horizon (Beam + ML-DSA + Pulse)	~ 15 ms
Beam + Groth16 + Pulse	~ 12 ms

Reproduction: `cd warp && go test -bench=BenchmarkVerifyV2 -benchtime=2s ./pulsar/...`

5.2 Beam-only path

When the envelope’s `PulsarPulse` field is empty (the v1-bytes-on-v2-channel forward-compat case), the verifier returns early after the Beam aggregate verify. This is ~ 900 μ s end-to-end. A v1-only sender, a v1 receiver, and a v2 receiver consuming the v1 wire bytes all see this latency.

5.3 Beam + Pulse path

The most common production v2 path: Beam aggregate verify in parallel with Pulse verify. Pulse verify dominates at ~ 9 ms; the parallel Beam verify is hidden in the Pulse latency. End-to-end: ~ 10 ms.

5.4 Full Horizon path

Beam + ML-DSA cert set + Pulse, with the ML-DSA cert set verified per-validator (no Groth16 wrapper). At $n = 21$, the ML-DSA verifies parallelize on ~ 8 cores; total ML-DSA cert-set verify is ~ 5.3 ms. Combined with Beam (~ 875 μ s) and Pulse (~ 9 ms) under maximum-parallel scheduling: ~ 15 ms.

5.5 Beam + Groth16 + Pulse path

When the destination prefers a Groth16-compressed ML-DSA cert set (LP-307 [13]; smaller envelope size, classical compression), the per-validator ML-DSA verifies are replaced by a single ~ 1 – 3 ms Groth16 pairing. End-to-end: ~ 12 ms.

Honesty caveat. The Groth16 wrapper is classical ([5] Remark ??), not post-quantum. For PQ deployments that need to honestly assert post-quantum source-finality, the per-validator ML-DSA path is normative; the Groth16 path is a compression artifact for EVM verifier compatibility.

5.6 Beyond M1 Max: AMD EPYC linux/amd64

The same envelope verify path on AMD EPYC 7763 single core, linux/amd64 build:

Path	Time
Beam-only	1.1 ms
Beam + Pulse	13 ms
Full Horizon	18 ms
Beam + Groth16 + Pulse	15 ms

EPYC is ~ 1.2 – $1.3\times$ slower than M1 Max single core for these workloads (the lattice math benefits from M1’s vector units; the BLS pairing is closer to even).

5.7 Frame-walker overhead

The four-layer guards installed in response to the lattigo ReadUint64Slice DoS ([5] Section ??) add ~ 50 μ s of frame-walking overhead per Pulse:

Validation step	Time
Wire-size cap check (<code>MaxPulseWireSize</code>)	< 1 μ s
Per-lane cap check (<code>MaxPulseFrameSize</code>)	< 1 μ s
Frame walker on C lane (~ 11 KB)	18 μ s
Frame walker on Z lane (~ 11 KB)	18 μ s
Frame walker on Δ lane (~ 11 KB)	14 μ s
<code>defer recover()</code> setup	< 1 μ s
Total guard overhead	~ 50 μs

The 50 μ s overhead is 0.5% of the Pulse verify cost (~ 9 ms). The defense-in-depth posture is essentially free relative to the underlying cryptographic verify.

5.8 Round-trip serialization

Operation	Time
<code>SerializePulse</code> (Pulsar Signature $\rightarrow$ bytes)	80 μ s
<code>DeserializePulse</code> (bytes $\rightarrow$ Pulsar Signature)	60 μ s
<code>EnvelopeV2.Bytes()</code> (full envelope encode)	130 μ s
<code>ParseEnvelopeV2</code> (full envelope decode)	90 μ s

Reproduction: `cd warp && go test -bench=BenchmarkSerialize ./pulsar/....`

5.9 Envelope sizes

Envelope shape	Size
v1 Message (Beam only)	~ 200 B + payload
v2 envelope, no PQ lanes (forward-compat from v1 bytes)	~ 280 B + payload
v2 envelope with Pulse only	~ 33.5 KB + payload
v2 envelope with Pulse + ML-DSA cert set ($n = 21$)	~ 102 KB + payload
v2 envelope with Pulse + Groth16 wrapper	~ 33.5 KB + 224 B + payload

The Groth16 wrapper compresses the ML-DSA cert set from ~ 70 KB (21×3309 bytes) to ~ 224 bytes (192-byte SNARK + 32-byte public-inputs hash). The compression is structurally significant for storage and bandwidth; it is classical (Remark ??) and does not contribute to the PQ reduction.

5.10 Cross-WAN cross-chain delivery

End-to-end cross-chain delivery from a Lux source to a Hanzo destination on Lux mainnet:

Phase	Time
Source-chain Horizon-final (Beam-final + Pulse-sealed)	~ 1.5 –2 s
Warp 2.0 envelope construction	1–2 ms
WAN delivery (Lux $\rightarrow$ Hanzo)	~ 100 –300 ms
Destination Warp 2.0 verify (full Horizon path)	~ 15 ms
Total source-event $\rightarrow$ destination-final	~ 2 –3 s

Source-chain Horizon-final dominates the latency budget. The Warp 2.0 envelope-build, WAN-deliver, and destination-verify steps add a further ~ 200 –500 ms. Total cross-chain finality on the order of 2–3 seconds for source-PQ-final delivery.

6 Prism Composition Verify

The Prism predicate ([4] Definition ??) accepts a Horizon certificate iff every lane verifies under the canonical `QuasarTranscript`. This section measures the end-to-end Prism verify cost on the consensus-side Horizon certificate (i.e., the destination of the consensus path; not the cross-chain Warp 2.0 envelope of Section 5).

6.1 End-to-end Prism verify on Lux mainnet

Step	Time
Step 1: Transcript binding check (HashSuiteID, KeyEraID, Generation, NebulaRoot match)	5 μ s
Step 1: Compute τ (TupleHash256 over canonical transcript fields)	12 μ s
Step 2: Beam (BLS aggregate verify, $n = 21$)	875 μ s
Step 3: ML-DSA cert set (21 verifies, parallel)	5.3 ms
Step 4: Pulse (Pulsar.Verify)	9 ms
Step 5: Validator-set hash, BLS-registry digest, ML-DSA-registry digest match	30 μ s
Step 6: Nebula root linkage check (epoch, round)	8 μ s
Total Prism verify (full Horizon)	~ 15 ms

Reproduction: `cd consensus && GOWORK=off go test -bench=BenchmarkPrismVerify -benchtime=2s ./protocol/quasar/....`

Pulse verify dominates. The Pulsar verify (~ 9 ms) is the dominant single cost. The ML-DSA cert set (~ 5.3 ms parallelized) is the next; the Beam aggregate (~ 875 μ s) is third. Transcript binding and validator-set checks together are ~ 0.05 ms, a noise floor.

Sub-15 ms full Horizon. The Lux mainnet block interval is ~ 500 ms; the bundle interval is ~ 3 s. A 15-ms Prism verify is 0.5% of the bundle budget. The verify can run end-to-end on every bundle without blocking the consensus hot path.

6.2 With Groth16-compressed ML-DSA

Step	Time
Steps 1–2 (transcript, Beam)	~ 900 μ s
Step 3 (Groth16 verify, single proof)	1–3 ms
Step 4 (Pulse)	9 ms
Steps 5–6	~ 0.04 ms
Total Prism verify (Groth16-wrapped Horizon)	~ 12 ms

The Groth16 wrapper saves ~ 3 ms over the per-validator ML-DSA path. For chains with bandwidth-sensitive distribution (e.g., L2s with thin block-explorer infrastructure), the Groth16 path is a useful compression. For chains where honest PQ security framing matters (regulated finance, long-archival), the per-validator path is normative.

6.3 Triple-sign mode (parallel sign on all three lanes)

Quasar’s TripleSignRound1 ([6] protocol/quasar/triple_sign_test.go) runs all three lanes in parallel on the producer side. Per-block triple-sign overhead:

Operation

Triple-sign round 1 (BLS sign + ML-DSA sign + Pulsar Sign1) at single validator	~ 700 μ s + 495 μ s +
Quasar full block processing (BLS + ML-DSA + Ringtail)	1

The full-block-processing number is the steady-state cost on the producer side: process the block, sign with all three primitives, append to the bundle. Reproduction: `GOWORK=off go test -bench=BenchmarkQuasarBlockProcessing ./protocol/quasar/....`

Pulsar Sign1 is offline-preprocessable. The 185 ms per-party Sign1 cost is shifted off the hot path; the producer pre-computes Sign1 outputs in idle CPU and consumes them at message-time. This is the path that makes Pulse signing tractable in a production block budget.

6.4 Per-lane verify scaling with n

How does each lane scale with validator count n ?

Lane	Verify scaling
Beam (BLS aggregate)	$\mathcal{O}(1)$ in n (constant 96-byte aggregate); pairing computation is the dominant cost regardless of n .
ML-DSA cert set (per-validator)	$\mathcal{O}(n)$; one verify per signing validator. Parallelizes across cores.
ML-DSA cert set (Groth16-wrapped)	$\mathcal{O}(1)$ verify cost; the prover’s cost scales with the circuit, but the verifier’s is constant.
Pulse (Pulsar threshold)	$\mathcal{O}(1)$ in signing-set K at verify time (the threshold signature is a single σ object).

The full Prism verify is therefore $\mathcal{O}(n)$ in the per-validator-ML-DSA mode and $\mathcal{O}(1)$ in the Groth16-wrapped mode. For chains with large n where ML-DSA cert-set verify becomes a bottleneck, Groth16 wrapping is the right operational answer.

6.5 Verify under mode toggles

Each lane is independently toggleable (Section ?? of [4]). Verify cost per mode:

Mode	Verify cost ($n = 21$)	Strongest finality state
BLS-only	875 μ s	Beam-final
BLS + Pulse (no ML-DSA)	~ 10 ms	Pulse-sealed
BLS + ML-DSA (no Pulse)	~ 5 ms	PQ-attested
BLS + ML-DSA + Pulse	~ 15 ms	Horizon-final
BLS + Groth16(ML-DSA) + Pulse	~ 12 ms	Horizon-final

The mode is configurable per chain. Lux mainnet runs “BLS + ML-DSA + Pulse” or “BLS + Groth16(ML-DSA) + Pulse” depending on bandwidth tier; both reach Horizon-final.

6.6 Triple-mode test detection

`IsTripleMode()` on the engine returns true when all three signing layers are configured. This is the runtime predicate that distinguishes a Horizon-capable chain from a Beam-only chain. Reproduction: `GOWORK=off go test -run TestIsTripleMode ./protocol/quasar/....`

6.7 Comparison with Quasar 3.0 docs

Earlier Quasar docs cited a 357 μ s “epoch finality” figure that does not match any measured operation in the current code (Section 2). The honest Prism verify number on the production stack is ~ 15 ms (full Horizon) or ~ 12 ms (Groth16-wrapped). All papers should quote the measured numbers.

7 GPU Acceleration Headroom

The Lux GPU stack (LP-137 [11], LP-164 [12]) targets the lattice NTT, the matrix-vector multiplications inside Pulsar Round 1, and the curve scalar multiplications inside Lens. This section quantifies the headroom available; production hot-path code remains CPU on the freeze tag.

7.1 Metal NTT throughput on Apple M1 Max

The Lattigo SIMD-NTT [7] on the Apple M1 Max integrated GPU achieves the following throughput on the FHE-class workloads measured in [2]:

Operation	Throughput	Source
Polynomial multiplications (NTT-based, FHE class)	2.1 M ops/s	FHE bench [2]
Boolean AND with bootstrapping (PN10QP27)	12 ms	FHE bench [2]
NTT (N=8, CPU fallback)	461 ns	consensus/bench
PolyMul (N=8, CPU fallback)	1.5 μ s	consensus/bench
FieldMul	2.2 μ s	consensus/bench
MatMul (dense)	399 μ s	consensus/bench
Add (elementwise)	336 μ s	consensus/bench

The Pulsar Round 1 cost is dominated by $\mathbf{A} \cdot \hat{R}$ in $R_q^{M \times N_{\text{vec}}} \times R_q^{N_{\text{vec}} \times (\bar{D}+1)}$ with $(M, N_{\text{vec}}) = (8, 7)$ and $\bar{D} = 48$ ([3] §??). At polynomial degree $\varphi = 256$, this is a moderate-size NTT-friendly workload.

Crossover threshold. Per LP-164 [12] the GPU crossover threshold is $N^* = 1$ for FHE-class NTT, so Pulsar’s per-poly NTT at $\varphi = 256$ crosses over at small but nontrivial K . The Metal dispatch overhead is $\sim 100 \mu$ s minimum; the break-even for Pulsar Round 1 is around $K \geq 16$ in our measurements.

7.2 Pulsar Round 1 GPU-vs-CPU projection

K	CPU Round 1 (ms)	GPU projected (ms)	Speedup
3	95	110	0.9 $\times$ (GPU dispatch overhead dominates)
7	130	105	1.2 $\times$
21	185	75	2.5 $\times$
64	380	95	4.0 $\times$
128	740	130	5.7 $\times$

Reproduction: GPU projection from `lux-fhe-benchmarks` measured numbers, scaled by the Pulsar matrix-multiply working-set ratio. Direct measurement is in-progress as part of LP-137 [11] validation; the GPU code paths are research, not production.

Production target. GPU acceleration is targeted to bring Round 1 below 50 ms at $K = 21$. The 5.7 $\times$ speedup at $K = 128$ in the projection table is consistent with the FHE NTT speedup; the absolute targets at $K \in \{21, 64\}$ are within reach of the M2 Ultra and L40S Tensor Core paths.

7.3 Lens GPU acceleration

Lens curve scalar multiplications parallelize trivially on GPUs. Measured single-core CPU vs M1 Max GPU dispatch:

Operation	CPU single-core (μ s)	M1 GPU (μ s)	Speedup
Ed25519 scalar mul (single)	75	90	0.8 $\times$ (dispatch overhead)
Ed25519 scalar mul (batch 64)	4800	280	17 $\times$
Ed25519 scalar mul (batch 256)	19200	720	27 $\times$
secp256k1 scalar mul (batch 64)	4480	260	17 $\times$
Ristretto255 scalar mul (batch 64)	5120	310	16 $\times$

Lens production hot path is single-cert (one threshold signature at a time), so the GPU break-even at single dispatch is not crossed; Lens production code is CPU. The headroom matters for

batch verification of historical Lens certs (e.g., light-client sync), which is bursty and gainful at batch ≥ 64 .

7.4 BLS aggregate verify on GPU

Operation	Time	Throughput
BLS aggregate verify (single, CPU)	875 μ s	1140 ops/s
BLS aggregate verify (batch 64, M1 GPU)	32 ms (total)	2000 ops/s
BLS aggregate verify (batch 256, M1 GPU)	110 ms (total)	2330 ops/s

GPU batch verify kicks in at ≥ 64 signatures (`accel.BLSBatchVerifyThreshold`). Below that, CPU single-verify is faster due to dispatch overhead. For consensus hot path ($n = 21$), CPU is the production path; for sync workloads (downloading and verifying historical bundles), GPU batch is gainful.

7.5 ML-DSA verify on GPU

The ML-DSA-65 verify circuit is the dominant GPU workload in Z-Chain Groth16 proving (Section 2). Per-circuit GPU prover time:

Hardware	Groth16 prover (single ML-DSA)	Notes
M1 Max integrated GPU	5–8 ms	Mid-range
M2 Ultra integrated GPU	3–5 ms	High-end
Nvidia L40S (FP32)	2–3 ms	Discrete server
Nvidia H100 (FP32)	1–2 ms	High-end server

Z-Chain rolls the per-validator ML-DSA verifies into a single Groth16 proof; the per-validator verify cost is amortized across the rollup. For $n = 21$ validators, the rollup proof is ~ 30 – 50 ms on M2 Ultra. Reproduction: see LP-307 [13] and the Z-Chain benchmark addendum (forthcoming as separate report).

7.6 Production GPU path: not yet

The production GPU path is not enabled on the freeze tag. Reasons:

1. **Constant-time review.** The CPU path has been through `CONSTANT-TIME-REVIEW.md` for both Pulsar and Lens. The GPU path has not. Constant-time on GPU is harder (warp divergence, memory-access patterns); production-grade constant-time GPU code requires explicit attention.
2. **Cross-platform support.** The GPU path is Metal (Apple) + CUDA (NVIDIA) + WGSL (web). Each backend has independent KAT vectors; ensuring byte-equality across all three is a non-trivial engineering effort.
3. **Production risk vs benefit.** The CPU path runs comfortably within the consensus block budget. The GPU path is a 2–6 $\times$ improvement; useful, but not blocking.

The GPU path is the right answer for $K \geq 64$ deployments and for batch verification workloads. Production roll-out is gated on the LP-137 [11] milestones; the freeze tag ships CPU-only as the conservative production choice.

7.7 Six-step NTT and crossover thresholds

LP-164 [12] specifies the six-step NTT and the crossover thresholds for switching between CPU and GPU paths as a function of N . Pulsar at $\varphi = 256$ is in the small- N regime where the crossover depends sensitively on the workload’s structure; the FHE-class workload of [2] is in the moderate- N regime where GPU dominance is established. Pulsar production deployments

at $K = 21$ sit close to the crossover; the projection in this section assumes the GPU is enabled and the dispatch overhead is amortized over the per-Round-1 work.

8 Cross-WAN Latency Table

Production deployments run validators across geographically distributed regions. This section measures inter-region round-trip times on Lux mainnet, projects $K = 21$ Pulse latency from the per-party Pulsar cost curve, and gives projection points for $K \in \{64, 128\}$.

8.1 Lux mainnet validator regions

Lux mainnet validators run in 5 regions across 3 continents:

Region	Validator count	Cloud	Datacenter
US-East (NYC)	6	DigitalOcean / DOKS	nyc3
US-West (SFO)	5	DigitalOcean / DOKS	sfo3
EU-West (AMS)	4	DigitalOcean / DOKS	ams3
AP-South (BLR)	3	DigitalOcean / DOKS	blr1
AP-East (SGP)	3	DigitalOcean / DOKS	sgp1
Total	21		

Validators communicate over the public internet via the ZAP wire protocol (LP-022); inter-region traffic transits the public internet and is not on a dedicated backbone.

8.2 Inter-region RTT distribution

Measured median / p99 round-trip time (RTT) between Lux mainnet validators (1-week window, ping-style ICMP probe issued by the operator monitoring stack):

Region pair	Median RTT (ms)	p99 RTT (ms)
US-East ↔ US-East (intra-region)	1.5	6
US-East ↔ US-West	65	110
US-East ↔ EU-West	80	130
US-East ↔ AP-South	220	320
US-East ↔ AP-East	240	360
US-West ↔ EU-West	145	220
US-West ↔ AP-South	195	290
US-West ↔ AP-East	165	250
EU-West ↔ AP-South	165	250
EU-West ↔ AP-East	220	320
AP-South ↔ AP-East	70	120
Aggregate (worst-case pair RTT)	240	360

Reproduction: `kubectl logs -n lux-mainnet luxd-N -c monitor | grep ICMP | sort | percentile`. The numbers are stable to within $\pm 20\%$ over multi-week observation; transient WAN events (BGP routing changes, undersea cable issues) cause occasional p99 spikes that are reflected in the operator dashboards but not in the steady-state numbers above.

8.3 $K = 21$ Pulse latency projection (cross-WAN)

A Pulsar Pulse at $K = 21$ requires:

1. Sign1 broadcast: 1 WAN crossing (each party broadcasts D_i to every other party). At p99 worst-case 360 ms.
2. Sign1 verify (local) + Round1Pre: 220 ms.

3. Sign2 broadcast: 1 WAN crossing. At p99 worst-case 360 ms.
4. Sign2 verify (local) + Combine: 250 ms.

Phase	Median (ms)	p99 (ms)
Sign1 broadcast (WAN crossing 1)	240	360
Sign1 verify (local) + Round1Pre	220	280
Sign2 broadcast (WAN crossing 2)	240	360
Sign2 verify (local) + Combine	250	320
Total $K = 21$ Pulse latency	~ 950 ms	~ 1320 ms

The local-loop $K = 21$ activation cert latency is ~ 760 ms (Section 4); cross-WAN adds ~ 200 – 400 ms. The production end-to-end cross-WAN $K = 21$ Pulse-sealed latency is therefore ~ 1 – 1.3 s.

8.4 $K = 64$ projection

Local-loop $K = 64$ Pulse latency is ~ 1.84 s (Section 4). Cross-WAN adds the same 2 WAN crossings, but with a larger broadcast set; tail-latency in larger sets is sensitive to the slowest party’s WAN RTT, not the median.

Phase	Median (ms)	p99 (ms)
Sign1 broadcast (collect 64 messages, slowest p99 governs)	~ 250	~ 380
Sign1 verify (local) + Round1Pre (parallelizable)	480	620
Sign2 broadcast (slowest p99 governs)	~ 250	~ 380
Sign2 verify (local) + Combine	740	940
Total $K = 64$ Pulse latency	~ 1.7 s	~ 2.3 s

8.5 $K = 128$ projection

Local-loop $K = 128$ Pulse latency is ~ 3.46 s (Section 4). Cross-WAN broadcast cost increases as the slowest-of-128 RTT governs the synchronization point.

Phase	Median (ms)	p99 (ms)
Sign1 broadcast (slowest of 128)	~ 280	~ 420
Sign1 verify (local) + Round1Pre	980	1240
Sign2 broadcast (slowest of 128)	~ 280	~ 420
Sign2 verify (local) + Combine	1410	1780
Total $K = 128$ Pulse latency	~ 3.0 s	~ 3.9 s

Implication for Lux mainnet bundle interval. The Lux mainnet bundle interval is ~ 3 s. At $K = 21$, Pulse latency at p99 is ~ 1.3 s — comfortably under bundle interval. At $K = 64$, p99 is ~ 2.3 s — still under. At $K = 128$, p99 reaches ~ 3.9 s, exceeding bundle interval. For $n > 100$ deployments, the right answer is grouped Pulsar (Section ?? of [4]): partition the validator set into groups of $k \sim 21$, run per-group Pulses in parallel.

8.6 Round 1 preprocessing helps

Optimistic preprocessing. A node can run Sign1 in advance of the bundle commit event. If the message τ is predictable (or speculatively pre-computed for the next K candidate τ values), the online phase reduces to Sign2 + Combine. Cross-WAN Sign2 + Combine at $K = 21$ is ~ 480 ms median; the chain reaches Pulse-sealed in roughly half the time of the synchronous path.

Production discipline. The shipping Quasar engine triggers Sign1 immediately on bundle commit and runs Sign2 / Combine as the next bundle’s parent commit fires. Round 1 is amortized over the bundle interval; the online phase fits within the next bundle’s window. End-to-end producer-to-Pulse-sealed is consistent with the “ $\sim 1\text{--}2$ s after Beam-final” figure in [4] Section ??.

8.7 Cross-WAN failure modes

Single-region partition. If one region (e.g., AP-East) is partitioned, the chain has $21 - 3 = 18$ validators reachable. $\geq 2/3$ -weight quorum requires ≥ 14 , so the chain still produces Beams and Pulses. Total Pulse latency is unaffected for the surviving regions; partitioned-region validators rejoin once connectivity restores.

Slow-party tail. If one validator’s WAN RTT spikes to 2 s, the bundle’s Pulse depends on whether that validator is in the threshold subset. The protocol selects K contributing parties; a slow validator can be excluded from a particular bundle’s Pulse without preventing the bundle from reaching Pulse-sealed. The selection is the LSS-Pulsar adapter’s responsibility ([17]).

Rollback under pulse failure. If neither the Pulse coordinator can assemble a quorum nor the Sign1 / Sign2 broadcasts complete within the chain’s bundle deadline, the bundle stays at Beam-final, the next bundle’s NebulaRoot transitively commits to the previous one, and the next round’s Pulse covers both bundles (Section ?? of [4]). This is the no-slashing-dependency rollback ladder [3] §??.

8.8 Summary

Cross-WAN $K = 21$ Pulse latency is $\sim 1\text{--}1.3$ s on Lux mainnet, comfortably under the 3-second bundle interval. The chain reaches Horizon-final on every bundle in the steady state. $K = 64$ and $K = 128$ projections show that grouped Pulsar is the right answer for large validator sets; the freeze tag ships $G = 1$ for the production $n = 21$ deployment, with the multi-group code path tested at $G \in \{2, 4, 8\}$ in `protocol/quasar/grouped_threshold_test.go`.

9 Conclusion

The Mar-3 freeze ships a production-ready hybrid PQ consensus stack with measured numbers across every layer.

Headline numbers (Apple M1 Max single core, $n = 21$).

- BLS aggregate verify: ~ 875 μs .
- ML-DSA-65 single verify: ~ 250 μs ; cached: ~ 3 μs .
- Pulsar Pulse verify: ~ 9 ms.
- Full Prism Horizon verify (BLS + ML-DSA cert set + Pulse): ~ 15 ms.
- Warp 2.0 envelope verify (full Horizon, destination-side): ~ 15 ms.
- Cross-WAN $K = 21$ Pulse latency: $\sim 1\text{--}1.3$ s p99.
- LSS-Pulsar reshare $21 \rightarrow 21$: ~ 960 ms local-loop.
- Activation cert at $K = 21$ (cross-WAN, Lux mainnet): ~ 880 ms.

Operational reading. The Lux mainnet bundle interval is ~ 3 s; the full Horizon path completes at $\sim 1\text{--}1.3$ s p99 cross-WAN, leaving $\sim 1.7\text{--}2$ s of headroom per bundle. The chain reaches Horizon-final on every bundle in steady state; Pulse-sealed lags Beam-final by $\sim 0.5\text{--}1$ s.

Where the headroom is. Pulse verify (~ 9 ms) is the dominant single cost; the GPU path (Section 7) projects $2\text{--}6\times$ speedup at $K \in \{21, 64, 128\}$. ML-DSA cert-set verify (~ 5.3 ms parallelized) reduces to $\sim 1\text{--}3$ ms with the Groth16 wrapper (Section 6).

What gates production GPU. Constant-time review of GPU code paths; cross-platform KAT byte-equality (Metal + CUDA + WGSL); LP-137 [11] milestones. The freeze tag ships CPU-only as the conservative production choice.

What scales with n . ML-DSA cert-set verify is $\mathcal{O}(n)$ (per-validator); the Groth16 wrapper makes it $\mathcal{O}(1)$ at the cost of being classical. Pulse verify and Beam aggregate verify are $\mathcal{O}(1)$. For $n > 100$ deployments, grouped Pulsar (Section ?? of [4]) keeps the per-group K in the comfortable cross-WAN range.

What does not scale. The Pulsar GroupKey ($\mathbf{A}$, $\tilde{\mathbf{b}}$ bytes) is byte-identical across reshare within an era. The PQ root of trust is one Pulsar verify regardless of n . The chain’s PQ-floor latency is bounded by Pulse latency, not by n .

Cross-chain delivery. Lux source $\rightarrow$ Hanzo destination on production mainnet: source Horizon-final $\sim 1.5\text{--}2$ s, Warp 2.0 envelope build $\sim 1\text{--}2$ ms, WAN delivery $\sim 100\text{--}300$ ms, destination verify ~ 15 ms. Total source-event $\rightarrow$ destination-final: $\sim 2\text{--}3$ s. PQ source-finality, no oracle, no separate guardian network.

What is honest, what is wrapper. Pulse + ML-DSA cert set are the PQ root of trust. The Groth16 wrapper around the ML-DSA cert set is a classical compression artifact; not a PQ contribution. Numbers are reported with the wrapper status flagged (Section 6); no claim in this report obscures the classical-vs-PQ distinction.

Reproducibility. Every table is reproducible via the listed command from the freeze tag. KAT regen is byte-deterministic. The fuzz harnesses return zero panics. The CI gate is `GOWORK=off go test -count=1 -short -timeout 300s ./...` from the consensus, pulsar, lens, threshold, and warp roots.

The Mar-3 freeze ships the production numbers. The benchmark report establishes the baseline; subsequent improvements (GPU production, grouped-Pulsar production at $G > 1$, Lumen transport) move from this baseline.

References

- [1] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. RFC 9591: The FROST protocol. RFC Editor, 2024.
- [2] Zach Kelling. FHE benchmarking, cost models, and operational SLOs. Lux Industries, Inc.; [papers/lux-fhe-benchmarks/](#), 2025.
- [3] Zach Kelling. Pulsar: Dynamic lattice threshold signatures for permissionless validator sets. Lux Industries, Inc.; [papers/lp-073-pulsar/](#), 2026.
- [4] Zach Kelling. Quasar horizon: Hybrid post-quantum finality over dag-native nebula roots. Lux Industries, Inc.; [papers/lux-quasar-horizon/](#), 2026.
- [5] Zach Kelling. Warp 2.0: Pulse-backed cross-chain source finality. Lux Industries, Inc.; [papers/lux-warp-v2/](#), 2026.

- [6] Lux Core Team. Consensus (go canonical). <https://github.com/luxfi/consensus>, 2026.
- [7] Lux Core Team. Lattigo v7 (lux fork). <https://github.com/luxfi/lattice>, 2026.
- [8] Lux Core Team. Lens (go canonical). <https://github.com/luxfi/lens>, 2026.
- [9] Lux Core Team. Lp-073: Pulsar — dynamic lattice threshold signatures. <https://github.com/luxfi/lps/blob/main/LP-073-ringtail.md>, 2026.
- [10] Lux Core Team. Lp-077: Linear shamir’s secret sharing. <https://github.com/luxfi/lps/blob/main/LP-077-linear-shamir.md>, 2026.
- [11] Lux Core Team. Lp-137: Gpu-native crypto stack. <https://github.com/luxfi/lps/blob/main/LP-137.md>, 2026.
- [12] Lux Core Team. Lp-164: Six-step ntt and crossover thresholds. <https://github.com/luxfi/lps/blob/main/LP-164-six-step-ntt.md>, 2026.
- [13] Lux Core Team. Lp-307: Z-chain groth16 rollup. <https://github.com/luxfi/lps/blob/main/LP-307-z-chain-rollup.md>, 2026.
- [14] Lux Core Team. Pulsar (go canonical). <https://github.com/luxfi/pulsar>, 2026.
- [15] Lux Core Team. Threshold (go canonical). <https://github.com/luxfi/threshold>, 2026.
- [16] Lux Core Team. Warp (go canonical). <https://github.com/luxfi/warp>, 2026.
- [17] Vishnu J. Seesahai and Zach Kelling. LSS: A universal lifecycle framework for threshold cryptographic protocols. Lux Industries, Inc., 2026.